#14 캠 정지 — 코드 상세 설명서

출처: Notion `[#14] 코드리뷰 · 캠 정지` (PR #116). 아래 코드는 Notion에 적힌 코드다.

1. 전체 흐름

```
관리자 화면: 캠 정지 버튼
   → CameraController.stop
   → CameraStopService.stop
        ① [커밋] 캠이 있고 안 지워졌나? 없으면 404 → run_requested = 0
        ② AnalyzerClient.stopCamera   POST /analyzer/v1/cameras/{id}/stop (15초)
              실패해도 예외를 던지지 않고 warning만 기록
        ③ [커밋] STOPPED, progress 0, 오류 내용 삭제
   → 200 {status: STOPPED, warning}  /  404
```

2. 명세가 요구한 것 vs 구현

명세 (FNC-A-021)	구현
1. run_requested=0	`Camera.cancelRunRequest()`, 1단계 트랜잭션에서 커밋
2. 분석 서버 `POST /analyzer/v1/cameras/{id}/stop` 호출	`AnalyzerClient.stopCamera` (X-Internal-Token, 타임아웃 15초)
3. status=STOPPED, progress=0	`markStopped` → `Camera.updateStatus(STOPPED, 0, null)` (이전 오류 내용도 지움)
4. 더미 캠: 마지막 녹화의 ended_at=현재 시각	**미구현** (recording이 dev에 없음, 호출 자리만 주석)
5. 열린 캠 이상 이벤트가 있으면 종료	**미구현** (이벤트 테이블 없음, 호출 자리만 주석)
6. 작업 기록(CAMERA_RUN, op: STOP)	**미구현** (#6 미병합, 호출 자리만 주석)
업무 규칙: 준비 중에 정지하면 분석 서버가 준비를 취소	준비 중이어도 같은 정지 지시를 보낸다 (취소는 분석 서버 몫)
예외: 분석 서버 연결 실패여도 STOPPED로 저장, 경고, 200	`RestClientException`을 잡아 `warning`에 담고 정상 진행
예외: run_requested=0이라 분석 서버가 다시 켜져도 가동하지 않음	1단계에서 먼저 확정 (호출보다 앞)
API: 200 `{"status": "STOPPED", "warning": ...}`, 오류 404	`CameraStopResponse`, `CameraNotFoundException`

3. 코드

3-1. Camera (common) 메서드 하나

```
/** 분석 서버가 이 캠을 가동하지 않아도 된다고 기록한다. 분석 서버가 다시 켜져도 가동하지 않는다(FNC-A-021 처리 순서 1). */
public void cancelRunRequest() {
    this.runRequested = false;
}
```

- #13의 `requestRun()`과 짝이다. 상태를 바꾸는 길을 메서드로만 열어 두었다. (setter 없음)
- 정지 상태 저장은 이미 있는 `updateStatus(STOPPED, 0, null)`을 쓴다. 세 번째 인자가 null이라 이전 오류 내용이 지워진다.

3-2. 응답 상자와 경비원

```
// CameraStopResponse.java
public record CameraStopResponse(CameraStatus status, String warning) {}

// CameraController에 추가
@PostMapping("/{id}/stop")
public CameraStopResponse stop(@PathVariable Long id) {
    return cameraStopService.stop(id);
}
```

`status`는 늘 `STOPPED`다. `warning`은 평소 `null`이고 전달에 실패했을 때만 문구가 들어간다. `null`이어도 응답에 `"warning": null`이 그대로 들어간다. (명세 예시와 같은 모양, 테스트로 확인)

3-3. 총무: CameraStopService (핵심)

```
@Service
@RequiredArgsConstructor
@Log4j2
public class CameraStopService {

    private static final String DELIVERY_FAILED_WARNING = "분석 서버에 전달 실패";
    private static final int STOPPED_PROGRESS = 0;

    private final CameraRepository cameraRepository;
    private final AnalyzerClient analyzerClient;
    private final TransactionTemplate transactionTemplate;

    public CameraStopResponse stop(Long cameraId) {
        // 처리 순서 1: run_requested=0. 분석 서버가 다시 켜져도 가동하지 않게 호출보다 먼저 확정한다
        transactionTemplate.executeWithoutResult(status -> cancelRunRequest(cameraId));

        // 처리 순서 2: 분석 서버에 정지 지시(타임아웃 15초). 준비 중이면 분석 서버가 준비를 취소한다
        String warning = null;
        try {
            analyzerClient.stopCamera(cameraId);
        } catch (RestClientException e) {
            log.warn("분석 서버에 캠 정지를 전달하지 못했습니다 cameraId={}: {}", cameraId, e.getMessage());
            warning = DELIVERY_FAILED_WARNING;
        }

        // 처리 순서 3: status=STOPPED, progress=0 (이전 오류 내용도 지운다)
        transactionTemplate.executeWithoutResult(status -> markStopped(cameraId));
        // 처리 순서 4·5·6은 필요한 테이블·함수가 dev에 들어오면 여기서 호출한다
        return new CameraStopResponse(CameraStatus.STOPPED, warning);
    }

    private void cancelRunRequest(Long cameraId) {
        cameraRepository.findByIdAndDeletedAtIsNull(cameraId)
                .orElseThrow(() -> new CameraNotFoundException(cameraId))
                .cancelRunRequest();
    }

    private void markStopped(Long cameraId) {
        cameraRepository.findByIdAndDeletedAtIsNull(cameraId)
                .ifPresent(camera -> camera.updateStatus(CameraStatus.STOPPED, STOPPED_PROGRESS, nul
```

```
l));
    }
}
```

부분	쉬운 뜻
메서드에 `@Transactional`이 없다	**일부러 뺐다** (#13과 같은 이유). 전체를 한 묶음으로 만들면 분석 서버를 최대 15초 기다리는 동안 DB 연결을 붙들고, 중간에 예외가 나면 앞의 변경까지 취소된다. 그래서 `TransactionTemplate`으로 ① run_requested=0 ② (호출은 묶음 밖) ③ 정지 기록을 따로 커밋한다
1단계에서 캠이 없으면 `CameraNotFoundException`	404. 아직 아무것도 바꾸지 않았고 분석 서버도 부르지 않는다
`catch (RestClientException e)`	연결 거부·응답 중 끊김·15초 초과·4xx/5xx 응답이 모두 이 예외로 모인다. 예외를 다시 던지지 않고 경고 문구로 바꾼다. 정지는 계속 진행한다
`warning = DELIVERY_FAILED_WARNING`	명세가 정한 문구 '분석 서버에 전달 실패'
3단계 `updateStatus(STOPPED, 0, null)`	정지 상태·진행률 0·오류 내용 삭제. 캠이 그 사이에 삭제됐다면(`ifPresent`) 조용히 넘어간다
상태를 먼저 확인하지 않는다	명세에 "STOPPED·ERROR만 가능" 같은 제한이 없다. 이미 정지된 캠에도 정지를 다시 지시해도 안전하다 (분석 서버 정지는 멱등)
`log.warn`	분석 서버 전달 실패는 WARN 로그

#13과 비교: #13은 실패하면 `catch`에서 `recordFailure` 후 `throw e`로 예외를 **다시 던졌다.** #14는 예외를 **먹고** 경고 문자열로 바꾼다. 그래서 응답이 502가 아니라 200이다.

3-4. 전화기: AnalyzerClient.stopCamera (이미 있던 것 그대로)

```
/** 분석 서버에 캠 정지를 지시한다(FNC-A-021). 실패하면 RestClientException을 던지고 호출하는 쪽이 정한다. */
public void stopCamera(Long cameraId) {
    stopClient.post().uri("/analyzer/v1/cameras/{id}/stop", cameraId).retrieve().toBodilessEntity();
}
```

- #13에서 팀원 #15의 임시 코드와 같은 이름으로 포함해 둔 메서드를 **그대로** 쓴다. (타임아웃 15초: 분석 서버가 종료를 최대 10초 기다리므로) 이번에는 주석만 정리했다.
- 팀원 #15의 엇갈림 정리도 이 메서드를 `RestClientException`으로 잡고 있어서, 던지는 예외 종류를 바꾸지 않았다.

4. 테스트 10개 (전부 통과)

가짜 분석 서버(`FakeAnalyzer`)는 가동 테스트(#13)와 같이 쓰도록 **공용 파일로 분리**했다. (`CameraStartTest`의 안쪽 클래스를 꺼냄, 가동 테스트 12개도 그대로 통과) 요청을 받는 순간 실행할 일(`onRequest`)을 정하는 기능을 추가했다.

테스트	확인하는 것
`stop_runningCamera_becomesStopped`	200 `{status: STOPPED, warning: null}`, DB: STOPPED·progress=0·run_requested=0

테스트	확인하는 것
`stop_sendsAnalyzerRequest`	분석 서버가 받은 요청: POST, 경로, X-Internal-Token, 호출 1회
`stop_preparingCamera_cancelsPreparation`	**완료 기준**: 준비 중(진행률 40) 캠을 정지하면 정지를 지시하고 STOPPED·progress=0·run_requested=0
`stop_errorCamera_clearsErrorMessage`	ERROR 캠(오류 내용 있음) 정지 → STOPPED, 오류 내용 지워짐
`stop_alreadyStopped_idempotent`	이미 정지된 캠도 200, 분석 서버에 정지 지시 (멱등)
`stop_cancelsRunRequestBeforeCallingAnalyzer`	분석 서버가 요청을 받는 순간 DB를 읽어 run_requested가 이미 0인지 확인 (처리 순서 1이 2보다 먼저)
`stop_analyzerConnectionFails_stillStoppedWithWarning`	연결이 끊겨도 200, warning='분석 서버에 전달 실패', 앱 쪽 STOPPED
`stop_analyzerErrorResponses_stillStoppedWithWarning`	분석 서버가 500·401·404로 답해도 200과 같은 경고, 앱 쪽 STOPPED
`stop_analyzerTimesOut_stillStoppedWithWarning`	분석 서버가 17초 뒤에 답하게 해 두고 약 15초(14.5~16.8초) 안에 경고와 함께 200으로 끝나는지 확인 (가장 느린 테스트, 약 15초)
`stop_unknownOrDeletedCamera_notFound`	없는 캠·삭제된 캠 → 404, 분석 서버 호출 0회, 삭제된 캠의 run_requested는 그대로

5. 왜 이렇게 만든 게 맞는가

결정	이유	근거
run_requested=0을 호출 전에 먼저 커밋	호출이 실패해도, 앱이 죽어도 "켜지 말라"는 기록이 남아 분석 서버가 다시 켜질 때 가동하지 않는다. 호출 시점에 이미 0인 것을 테스트로 고정	FNC-A-021 처리 순서 1
분석 서버 호출 실패를 예외로 던지지 않고 경고로 바꿈	명세가 "연결 실패여도 앱 쪽은 STOPPED로 저장, 경고 반환(200)"이라고 정했다	FNC-A-021 예외 상황
트랜잭션을 세 단계로 쪼갬 (#13과 같은 구조)	분석 서버를 최대 15초 기다리는 동안 DB를 붙들지 않고, 호출 결과와 상관없이 앞뒤 기록이 각각 확정된다	#13 코드리뷰 7번
이미 정지된 캠에도 같은 흐름을 그대로 실행	명세에 상태 제한이 없고 분석 서버 정지가 멱등이다. 상태가 엇갈린 캠을 정지로 정리하는 길도 된다	api/analyzer.md 정지(멱등)
`AnalyzerClient.stopCamera`를 바꾸지 않고 사용	팀원 #15가 같은 메서드를 쓰고 있어 예외 종류를 바꾸면 그쪽이 깨질 수 있다	#15 브랜치 `CameraStatusService`
가짜 분석 서버를 공용 파일로 분리	가동·정지 테스트가 같은 도구를 쓰고, 같은 코드를 두 번 쓰지 않는다	-

6. 선배가 물을 만한 질문과 답

질문	답
"분석 서버에 전달 못 했는데 왜 STOPPED로 저장해?"	명세가 그렇게 정했다. 관리자는 끄고 싶다는 뜻을 밝혔고, 분석 서버가 응답하지 않는다고 "켜짐"으로 남기면 끌 수 없는 카메라가 된다. 대신 경고를 돌려주고 `run_requested=0`을 먼저 확정해 둬서 분석 서버가 다시 켜져도 가동하지 않는다.
"#13은 실패하면 ERROR인데 #14는 왜 달라?"	가동은 "시작하지 못했다"가 실패라 ERROR로 알리고, 정지는 "끄려는 뜻"이 우선이라 앱 쪽을 정지로 확정한다. 둘 다 명세가 정했다.
"`@Transactional`이 왜 없어?"	#13과 같은 이유다. 분석 서버를 최대 15초 기다리는 동안 DB를 붙들지 않으려고 `TransactionTemplate`으로 세 단계로 나눠 커밋한다.
"이미 정지된 캠을 또 정지하면?"	200이고 분석 서버에도 정지를 지시한다. 멱등이라 안전하고, 앱과 분석 서버 상태가 엇갈린 캠을 정리하는 길도 된다.
"처리 순서 4·5·6은 왜 없어?"	recording 엔티티는 팀원 #17에만, 이벤트 테이블은 어디에도, 작업 기록은 #6의 PR #113(미병합)에 있다. 호출 자리만 주석으로 남겼다. 이 PR만으로는 완료 기준을 다 채우지 못한다고 PR에도 적었다.
"작업 기록(#113)을 나중에 연결할 때 주의할 점은?"	`WorkHistoryService`는 트랜잭션 없이 부르면 거부한다. 3단계(정지 기록) 트랜잭션 안에서 호출해야 한다.
"401이든 500이든 같은 경고면 원인을 어떻게 알아?"	지금은 구분이 안 된다. 내가 정한 것이라 leads와 문구를 나눌지 정하자고 제안했다. 서버 로그(WARN)에는 남는다.
"타임아웃 테스트가 15초나 걸리면 CI가 느려지지 않아?"	맞다. 타임아웃을 설정값으로 빼면 테스트에서 짧게 줄일 수 있다. 지금은 명세의 값(15초)을 그대로 검증하는 쪽을 택했다.

7. 명세 대조 (FNC-A-021)

명세 항목	구현	확인
1. run_requested=0	`cancelRunRequest`	`stop_cancelsRunRequestBeforeCallingAnalyzer`, `stop_runningCamera_becomesStopped`
2. 분석 서버 stop 호출	`AnalyzerClient.stopCamera`	`stop_sendsAnalyzerRequest`, `stop_analyzerTimesOut_stillStoppedWithWarning`
3. status=STOPPED, progress=0	`markStopped`	`stop_runningCamera_becomesStopped`, `stop_errorCamera_clearsErrorMessage`
4. 녹화 종료(더미 캠)	**미구현** (recording 없음)	-
5. 열린 이상 이벤트 종료	**미구현** (이벤트 테이블 없음)	-
6. 작업 기록	**미구현** (#6 미병합)	-

명세 항목	구현	확인
예외: 분석 서버 연결 실패 → STOPPED 저장, 경고, 200	`catch RestClientException` → `warning`	`stop_analyzerConnectionFails_...`, `stop_analyzerErrorResponses_...`, `stop_analyzerTimesOut_...`
오류 응답: 404	`CameraNotFoundException`	`stop_unknownOrDeletedCamera_notFound`
완료 기준: 준비 중 정지 시 준비 취소	준비 중에도 같은 정지 지시 (취소는 분석 서버)	`stop_preparingCamera_cancelsPreparation`
후속: 캠 상태 SSE	이슈 범위 밖 (2단계)	-